

ĐỒ ÁN MÔN HỌC – UniHub Workshop

[View on GitHub](#)

ĐỒ ÁN MÔN HỌC – UniHub Workshop

Bối cảnh

Trường Đại học A tổ chức **“Tuần lễ kỹ năng và nghề nghiệp”** hàng năm. Sự kiện kéo dài 5 ngày, mỗi ngày có 8–12 workshop diễn ra song song tại nhiều phòng khác nhau. Hiện tại ban tổ chức quản lý đăng ký bằng Google Form và thông báo qua email thủ công — quy trình này không còn đáp ứng được nhu cầu khi quy mô ngày càng lớn.

Ban tổ chức muốn xây dựng hệ thống **UniHub Workshop** để số hóa toàn bộ quy trình, từ đăng ký đến check-in tại sự kiện.

Người dùng

Nhóm	Mô tả
Sinh viên	Xem lịch workshop, đăng ký, nhận xác nhận, check-in khi tham dự
Ban tổ chức	Tạo và quản lý workshop, theo dõi số lượng đăng ký
Nhân sự check-in	Xác nhận sinh viên tham dự tại cửa phòng bằng mobile app

Yêu cầu hệ thống

Xem và đăng ký workshop

Sinh viên có thể xem danh sách tất cả workshop trong tuần lễ, bao gồm thông tin diễn giả, phòng tổ chức, sơ đồ phòng và số chỗ còn lại theo thời gian thực. Sinh viên đăng ký tham dự workshop — một số workshop miễn phí, một số có thu phí. Sau khi đăng ký thành công, sinh viên nhận mã QR dùng để check-in.

Thông báo

Sau khi đăng ký thành công, sinh viên nhận thông báo xác nhận qua app và email. Hệ thống cần thiết kế để dễ dàng bổ sung kênh thông báo mới (ví dụ: Telegram) trong các học kỳ sau mà không cần thay đổi lớn.

Quản trị

Ban tổ chức dùng trang web admin để tạo workshop mới, cập nhật thông tin, đổi phòng, đổi giờ hoặc hủy workshop. Trang admin chỉ dành cho nội bộ và cần kiểm soát truy cập chặt chẽ — ba nhóm người dùng có quyền hạn khác nhau: sinh viên chỉ có thể xem và đăng ký workshop; ban tổ chức có quyền tạo, sửa, hủy workshop và xem thống kê; nhân sự check-in chỉ có quyền truy cập chức năng quét mã QR.

Check-in tại sự kiện

Nhân sự tại cửa phòng dùng mobile app để quét mã QR của sinh viên. Một số khu vực trong trường có kết nối mạng không ổn định — app phải cho phép ghi nhận check-in tạm thời khi không có mạng và tự đồng bộ lại khi kết nối được phục hồi.

AI Summary

Ban tổ chức có thể tải lên file PDF giới thiệu về workshop. Hệ thống tự động xử lý, tách nội dung, làm sạch văn bản và gửi sang mô hình AI để tạo bản tóm tắt hiển thị trên trang chi tiết workshop.

Đồng bộ dữ liệu sinh viên

Hệ thống quản lý sinh viên hiện tại của trường chưa có API. Cách duy nhất để lấy dữ liệu là qua file CSV mà hệ thống cũ export vào ban đêm. UniHub Workshop cần định kỳ nhập dữ liệu này để xác thực sinh viên khi đăng ký.

Các vấn đề cần giải quyết

Tranh chấp chỗ ngồi: Một số workshop chỉ có 60 chỗ nhưng có thể có hàng trăm sinh viên cố đăng ký cùng lúc ngay khi mở đăng ký. Hệ thống phải đảm bảo không có hai sinh viên nào cùng nhận được chỗ cuối cùng.

Tải trọng đột biến: Dự kiến khoảng 12.000 sinh viên truy cập trong 10 phút đầu khi mở đăng ký, trong đó 60% dồn vào 3 phút đầu tiên. Hệ thống cần có cơ chế bảo vệ backend API khỏi bị quá tải,

ngăn chặn các client gửi request liên tục và đảm bảo tính công bằng giữa các sinh viên đăng ký.

Thanh toán không ổn định: Nếu cổng thanh toán gặp sự cố, sinh viên vẫn phải xem được lịch workshop và thông tin sự kiện bình thường. Luồng đăng ký có phí cần xử lý tình huống thanh toán timeout mà không gây ra trừ tiền hai lần, đồng thời các tính năng không liên quan đến thanh toán vẫn phải hoạt động bình thường khi cổng thanh toán gặp sự cố kéo dài.

Check-in offline: Nhân sự ở khu vực mất mạng vẫn phải check-in được cho sinh viên; dữ liệu không được mất khi kết nối trở lại.

Tích hợp một chiều: Không thể gọi API hệ thống cũ — chỉ có thể đọc CSV được export theo lịch cố định. Luồng nhập dữ liệu phải xử lý được file lỗi, dữ liệu trùng và không làm gián đoạn hệ thống đang chạy.

Các nội dung cần thực hiện

Phần 1 — Blueprint

1. Tài liệu thiết kế hệ thống

Mô tả kiến trúc tổng thể của hệ thống, bao gồm các thành phần chính, cách chúng giao tiếp và lý do lựa chọn kiến trúc đó. Tài liệu cần trả lời được các câu hỏi: hệ thống gồm những phần nào, phần nào nói chuyện với nhau như thế nào, và khi một phần gặp sự cố thì phần còn lại bị ảnh hưởng ra sao.

2. C4 Diagram

Vẽ hai cấp độ đầu của C4 diagram:

- **Level 1 – System Context:** thể hiện UniHub Workshop trong bức tranh toàn cảnh — ai dùng hệ thống, hệ thống ngoài nào được tích hợp.
- **Level 2 – Container:** phân rã hệ thống thành các container (ví dụ: web app, mobile app, backend API, database, message broker), chỉ rõ công nghệ đề xuất và cách các container giao tiếp với nhau.

3. High-Level Architecture Diagram

Vẽ sơ đồ kiến trúc tổng quan thể hiện luồng dữ liệu và sự phụ thuộc giữa các thành phần, đặc biệt ở các điểm tích hợp (hệ thống cũ, cổng thanh toán, AI model) và luồng check-in offline.

4. Thiết kế cơ sở dữ liệu

Xác định các loại dữ liệu chính trong hệ thống, đề xuất loại database phù hợp (SQL, NoSQL, hoặc kết hợp) và giải thích lý do lựa chọn dựa trên đặc điểm của từng loại dữ liệu. Thiết kế schema cho các entity quan trọng nhất.

5. Mô tả các luồng nghiệp vụ quan trọng

Mô tả chi tiết ít nhất hai trong số các luồng sau:

- Luồng đăng ký workshop có phí (từ khi bấm “Đăng ký” đến khi nhận mã QR)
- Luồng check-in khi mất mạng và đồng bộ lại
- Luồng nhập dữ liệu từ CSV đêm

Với mỗi luồng, trình bày các bước xử lý, các thành phần tham gia và cách hệ thống phản ứng khi có lỗi xảy ra giữa chừng.

6. Thiết kế kiểm soát truy cập

Thiết kế mô hình phân quyền cho hệ thống. Xác định các nhóm người dùng, quyền hạn tương ứng với từng nhóm, và giải thích cách hệ thống kiểm tra quyền tại từng điểm truy cập (API endpoint, trang admin, mobile app). Nhóm có thể tham khảo mô hình **RBAC (Role-Based Access Control)** hoặc đề xuất cách tiếp cận khác nếu có lý do phù hợp.

7. Thiết kế các cơ chế bảo vệ hệ thống

Với mỗi vấn đề kỹ thuật dưới đây, trình bày giải pháp nhóm lựa chọn, giải thích cách nó hoạt động và lý do phù hợp với bài toán. Các kỹ thuật được gợi ý nhưng nhóm có thể đề xuất giải pháp thay thế nếu lập luận thuyết phục:

- **Kiểm soát tải đột biến:** Làm thế nào để backend API không bị quá tải khi 12.000 sinh viên đăng ký cùng lúc? (gợi ý: *Rate Limiting — Fixed Window, Sliding Window, Token Bucket, Leaky Bucket*)
- **Xử lý cổng thanh toán không ổn định:** Làm thế nào để hệ thống phản ứng khi cổng thanh toán liên tục lỗi mà không kéo sập toàn bộ dịch vụ? (gợi ý: *Circuit Breaker với các trạng thái Closed / Open / Half-Open, kết hợp Graceful Degradation*)
- **Chống trừ tiền hai lần:** Làm thế nào để đảm bảo một giao dịch chỉ được thực hiện đúng một lần dù client retry nhiều lần? (gợi ý: *Idempotency Key — cơ chế sinh key, nơi lưu trữ, cách kiểm*

tra trùng lặp, thời gian hết hạn)

Phần 2 — Cài đặt

Phần mềm hoàn chỉnh, có thể chạy được, cài đặt toàn bộ hệ thống đã mô tả trong Blueprint. Phần cài đặt phải bao gồm:

- **Tính năng nghiệp vụ đầy đủ:** Tất cả các chức năng được mô tả trong phần Yêu cầu hệ thống — xem và đăng ký workshop, thông báo, quản trị, check-in, AI Summary, đồng bộ CSV.
- **Các cơ chế kỹ thuật:** Toàn bộ giải pháp đã thiết kế trong Blueprint mục 6 và 7 phải được cài đặt thực sự trong code, không chỉ mô phỏng hoặc stub.
- **Hướng dẫn khởi chạy:** README rõ ràng, đủ để người chấm có thể clone repository và chạy được hệ thống mà không cần hỏi thêm.
- **Dữ liệu mẫu:** Seed data hoặc script tạo dữ liệu ban đầu để có thể thao tác và kiểm tra ngay sau khi khởi chạy.

Tham khảo: Template Blueprint

Template tham khảo theo cấu trúc của [OpenSpec](#) — framework spec-driven development, gồm ba lớp tài liệu: **proposal** (vấn đề và lý do), **design** (giải pháp kỹ thuật), **specs** (kịch bản và ràng buộc cho từng tính năng). Nhóm có thể bổ sung mục hoặc điều chỉnh cấu trúc nếu phù hợp.

```
blueprint/  
├─ proposal.md      # Bối cảnh, vấn đề, mục tiêu  
├─ design.md        # Kiến trúc, sơ đồ, quyết định kỹ thuật  
└─ specs/  
    ├─ auth.md      # Đặc tả phân quyền  
    ├─ payment.md    # Đặc tả luồng thanh toán  
    ├─ checkin.md    # Đặc tả luồng check-in offline  
    └─ ...           # Đặc tả các tính năng khác
```

proposal.md

```
# UniHub Workshop – Project Proposal  
  
## Vấn đề  
<!-- Mô tả vấn đề hiện tại mà hệ thống cần giải quyết.  
      Tại sao Google Form không còn đủ? Hậu quả cụ thể là gì? -->
```

```
## Mục tiêu
<!-- Hệ thống cần đạt được gì? Định lượng nếu có thể.
      Ví dụ: hỗ trợ 12.000 sinh viên đăng ký trong 10 phút đầu. -->

## Người dùng và nhu cầu
<!-- Ai dùng hệ thống? Họ cần làm gì? Điều gì quan trọng nhất với họ? -->

## Phạm vi
<!-- Những gì thuộc phạm vi đồ án này.
      Những gì KHÔNG thuộc phạm vi (ví dụ: payment gateway thật, hạ tầng production). -->

## Rủi ro và ràng buộc
<!-- Các vấn đề kỹ thuật đã biết trước: tranh chấp chỗ ngồi, tải đột biến,
      cổng thanh toán không ổn định, check-in offline, tích hợp một chiều CSV. -->
```

design.md

```
# UniHub Workshop – Technical Design

## Kiến trúc tổng thể
<!-- Mô tả architectural style được chọn và lý do.
      Hệ thống gồm những thành phần nào? Chúng giao tiếp với nhau như thế nào? -->

## C4 Diagram

### Level 1 – System Context
<!-- Sơ đồ: UniHub Workshop + actors + hệ thống ngoài -->

### Level 2 – Container
<!-- Sơ đồ: web app, mobile app, backend API, database, message broker, ... -->

## High-Level Architecture Diagram
<!-- Sơ đồ luồng dữ liệu, đặc biệt tại các điểm tích hợp và luồng check-in offline -->

## Thiết kế cơ sở dữ liệu
<!-- Loại database, lý do lựa chọn, schema các entity chính -->

## Thiết kế kiểm soát truy cập
<!-- Mô hình phân quyền, các nhóm người dùng, cách kiểm tra quyền tại từng điểm truy cập -->

## Thiết kế các cơ chế bảo vệ hệ thống

### Kiểm soát tải đột biến
<!-- Giải pháp, thuật toán, ngưỡng, hành vi khi vượt ngưỡng -->
```

```

### Xử lý cổng thanh toán không ổn định
<!-- Giải pháp, các trạng thái, ngưỡng kích hoạt, hành vi khi lỗi -->

### Chống trừ tiền hai lần
<!-- Cơ chế, nơi lưu trữ, TTL, luồng xử lý khi phát hiện trùng lặp -->

## Các quyết định kỹ thuật quan trọng (ADR)
<!-- Với mỗi quyết định lớn: lựa chọn gì, tại sao, đánh đổi gì.
    Ví dụ: SQL vs NoSQL, JWT vs Session, Kafka vs RabbitMQ, ... -->

```

specs/[feature].md

```

# Đặc tả: [Tên tính năng]

## Mô tả
<!-- Tính năng này làm gì? -->

## Luồng chính
<!-- Các bước xử lý theo thứ tự, các thành phần tham gia -->

## Kịch bản lỗi
<!-- Điều gì xảy ra khi: timeout, mất mạng, dữ liệu không hợp lệ, ... -->

## Ràng buộc
<!-- Giới hạn hiệu năng, bảo mật, tính nhất quán cần đảm bảo -->

## Tiêu chí chấp nhận
<!-- Làm thế nào để biết tính năng này hoạt động đúng? -->

```

Quy định nộp bài

Định dạng file nộp

- Mỗi nhóm nộp một file text duy nhất lên hệ thống.
- Tên file: mã-nhóm_mssv1_mssv2_mssv3_mssv4.txt (ví dụ: N01_21127001_21127002_21127003_21127004.txt).
- Nội dung file: **Link Google Drive** public chứa tất cả các thành phần bài làm.

Cấu trúc thư mục trên Google Drive

Thư mục Drive của nhóm phải bao gồm đủ ba thành phần:

1. **Blueprint** — Nhóm có thể nộp theo một trong hai hình thức:
 - **PDF:** Một file `blueprint.pdf` duy nhất chứa đầy đủ các thành phần theo template.
 - **Markdown:** Thư mục `blueprint/` tổ chức theo cấu trúc template, upload trực tiếp lên Drive.
2. **Source code** — Thư mục `src/` chứa toàn bộ mã nguồn, kèm thư mục `data/` chứa seed data và script khởi tạo cơ sở dữ liệu, và file `README.md` với hướng dẫn cài đặt và khởi chạy.
3. **Video trình bày** — Thư mục `clips/` chứa video quay màn hình trình bày các vấn đề kỹ thuật mà nhóm đã giải quyết (không cần slide). Nội dung phải bao gồm **camera thành viên thuyết trình** và **demo trực tiếp trên code hoặc ứng dụng đang chạy**. Quy định kỹ thuật: độ phân giải **FullHD (1080p)**, bitrate khoảng **720 kbps**, định dạng MP4.

software-design-docs is maintained by **nndkhoa**.

This page was generated by [GitHub Pages](#).